



UNIVERSIDAD AUTÓNOMA DE ENTRE RÍOS

FACULTAD DE CIENCIA Y TECNOLOGÍA

LIC. EN SISTEMAS DE INFORMACIÓN

ASIGNATURA

Programación orientada a objetos.

DOCENTES

- Ing. Tournour, Adrian.
- Perez, Alejandro.

INTEGRANTES DEL GRUPO

- Caire, Sofía.
- Pavan, Iara.
- Uriarte, Iara.

AÑO: 2026

Trabajo práctico 3 - Desarrollo y modelado de objetos.

Metodología y Uso de Inteligencia Artificial:

En este práctico deberán codificar en el lenguaje orientado a objetos de su preferencia (Java, C#, Python, etc.). Se fomenta el uso de herramientas de IA para asistir en la generación de código base, pero bajo directivas estrictas.

REGLA ARQUITECTÓNICA FUNDAMENTAL: prohibido mezclar lógica con interfaz.

Las clases de dominio (Entidades como Empleado, País, Cancha) y las clases de gestión puras NO deben tener NINGUNA interacción con el usuario. NO pueden contener comandos de impresión en pantalla (ej. System.out.println) ni lectura de teclado. Todo cálculo o validación debe retornar valores o booleanos, y será EXCLUSIVAMENTE la clase principal (Main o una clase UI/Controller) la encargada de mostrar los mensajes al usuario y leer sus ingresos.

REGLA DE PROMPTING OBLIGATORIA (Copiar y pegar a la IA): "Escribe el código en [Tu Lenguaje]. Utiliza exclusivamente conceptos básicos de Programación Orientada a Objetos. NO utilices manejo de excepciones, bases de datos ni librerías externas. Aplica una estricta separación de responsabilidades: las clases de entidad y lógica de negocio NO deben contener funciones de impresión en consola ni lectura de teclado; toda la interacción con el usuario debe ocurrir únicamente en la clase Main."

Ejercicio 1: La Clase Entero

Considere la siguiente clase Entero (ejemplo en Java):

```
public class Entero {  
    private int numero;  
  
    public Entero(int numero) {  
        this.numero = numero;  
    }  
    public int getNumero() {return numero; }  
    public void setNumero(int numero) {this.numero = numero; }  
  
    // Calcula el cuadrado de n  
    public long cuadrado() {return (long) numero * numero; }  
}
```

Consignas:

1. Agregue los siguientes métodos a la clase respetando el encapsulamiento: esPar(), esImpar(), factorial(), esPrimo().
2. Cree una clase Main para instanciar objetos y probar todos los métodos por consola.
3. **Realice el Diagrama de Clases (UML)** de la solución.

Ejercicio 2: Gestión de Empleados

Cree una aplicación por consola que gestione empleados.

1. La clase Empleado debe tener nombre, DNI y sueldo.

2. Regla de Negocio: El sueldo no puede ser negativo ni menor al Salario Mínimo Vital y Móvil actual. Si se intenta ingresar un valor menor, el método debe asignar automáticamente el valor del Salario Mínimo.
3. La clase Empresa (o GestorEmpleados) debe manejar una lista de empleados y tener métodos para:
 - Registrar nuevos empleados (validando que no se repita el DNI).
 - Devolver el objeto al Empleado que más gana (¡No imprimirlo, devolverlo!).
 - Devolver el sueldo promedio (double o float).
4. La clase Main debe interactuar con el usuario, llamar a los métodos de empresa y mostrar los resultados.
5. **Realice el Diagrama de Clases (UML)** identificando las relaciones entre Main, Empresa y Empleado.

Ejercicio 3: El Mapa Mundial Cree una aplicación que represente el mapa del mundo mediante objetos.

1. Diseño de Clases:
 - Continente: Tiene un nombre y una colección de países.
 - País: Tiene nombre, capital, superficie (en km²), pertenece a un Continente, tiene una colección de Provincias y una colección de Países Limítrofes.
 - Provincia: Tiene un nombre.
2. Requisitos de las Clases:
 - Constructores que inicialicen las colecciones vacías.
 - Métodos para vincular objetos: ej. agregarPais(Pais p), agregarProvincia(Provincia p), agregarLimitrofe(Pais p).
 - Sobrescribir el método toString() en todas las entidades para representar sus datos en texto.
3. Población de Datos con IA:
 - Consigna de IA: Pídanle a la Inteligencia Artificial que genere un método inicializarDatos() (o una clase Seeder) que contenga el código para instanciar todos los países de América y Europa, junto con sus respectivas provincias/estados/departamentos principales, y los vincule a sus continentes y establezca sus límites. Copien ese bloque de código en su proyecto para no tener que cargar datos a mano.
4. Interacción por Consola (Main): Presente un menú que permita:
 - Ingresar el nombre de un Continente y listar sus países.
 - Ingresar el nombre de un País y listar sus provincias.
 - Ingresar el nombre de un País y listar sus países limítrofes.
 - Listar todos los países ordenados por superficie (de mayor a menor).
 - Comparar 2 países por nombre e imprimir cuál tiene mayor superficie.
5. Realice el Diagrama de Clases (UML) de todo el modelo de dominio.

Ejercicio 4: Caso de Estudio Integrador - Complejo Deportivo

Un complejo deportivo tiene 3 canchas de Fútbol 5 y necesita un sistema para gestionar las reservas. Tienen la clase Cancha y la clase Turno.

Reglas de Negocio a implementar:

- El complejo abre de 14:00 a 23:00 hs. Solo se pueden crear turnos en esas horas en punto.
- Un Turno registra el nombre de la persona y la hora.
- **Separación de Responsabilidades:** La clase Cancha debe tener un método reservarTurno(Turno t). Este método debe validar que no exista ya una reserva para esa misma hora. **Si ya está ocupado, NO debe imprimir nada; debe retornar false.** Si la reserva es exitosa, se agrega a la lista y retorna true.

Requisitos de la Aplicación (Main):

1. Instanciar las 3 canchas.
2. Mostrar un menú interactivo que permita:
 - Ver el estado actual de las 3 canchas (horarios libres y ocupados).
 - Registrar una nueva reserva (la clase Main evaluará el true/false que devuelve la Cancha e imprimirá "Reserva exitosa" o "Error: Turno ocupado").
 - Cancelar una reserva existente.
3. **Realice el Diagrama de Clases (UML)** identificando las clases y sus relaciones.